

Builder Journal

Custom Object Detection System: Helmet Detection

1. Dataset Challenges

The project requirement called for a self-collected dataset of 300–1000 images with variation in background, lighting, angle, occlusion, and scale. To meet this within the available timeframe while still satisfying the self-collection requirement, a curated public base dataset (1,376 images, YOLO-format, 2 classes) was extended with self-captured and self-annotated images, merged into a single combined dataset via Roboflow rather than used as-is.

- Initial dataset export from Roboflow failed silently (0-byte file) due to an incomplete browser download; resolved by re-generating the dataset version and re-exporting.
- Captured and annotated a batch of original images (self and volunteers, with and without helmets, across different lighting and distances) and added them into the same Roboflow project to be merged with the base dataset on the next generated version.
- Verified the base dataset's train/valid/test split (1,185 / 127 / 64 images) and class balance (2,390 “With Helmet” vs. 1,228 “Without Helmet” instances, $\approx 1.95:1$ ratio) prior to merging, to track how the added images shift these numbers.

2. Annotation Challenges

Since the dataset arrived pre-annotated, the main challenge shifted from creating labels to verifying their completeness and accuracy before trusting them for training.

- Bounding boxes and label files are stored separately from images (YOLO .txt format), so annotations are not visible without writing a script to overlay them — this made an initial visual sanity check non-obvious.
- Ran a programmatic completeness check across all 1,376 images to confirm zero missing label files and zero empty label files.
- Performed a manual visual spot-check on a random 12-image sample (boxes rendered and inspected by eye) to confirm bounding box tightness and correct class assignment, including on images with multiple riders per frame.

3. Training Challenges

Training has not yet been executed for the helmet detection model at the time of writing; the following are the anticipated challenges going into Week 3, based on issues encountered in the earlier PPE compliance detection project using the same YOLOv8s pipeline.

- Class imbalance ($\approx 2:1$ in favor of “With Helmet”) may bias the model toward the majority class, requiring monitoring of per-class precision/recall rather than aggregate accuracy alone.
- Small, distant, or partially occluded riders are expected to be the hardest cases, consistent with known weaknesses of single-shot detectors on small objects.
- Local hardware constraints (no dedicated GPU) may require reduced batch size, a smaller YOLOv8 variant (n/s), or use of a cloud/Colab GPU runtime to keep training time reasonable.

4. Performance Improvements

Planned and applied strategies to improve model quality, carried over from lessons learned on the prior PPE detection project:

- Use pretrained COCO weights as the starting point (transfer learning) rather than training from scratch, given the moderate dataset size.
- Apply standard augmentation (mosaic, horizontal flip, brightness/contrast jitter) already built into the YOLOv8 training pipeline to improve robustness to the lighting and angle variation present in the dataset.
- Track validation mAP per epoch to catch overfitting early, given the training set is heavily weighted (86%) relative to validation and test.
- Reserve the test set strictly for final evaluation only, avoiding any tuning decisions based on test performance.

5. Lessons Learned

- Verifying a dataset (completeness + accuracy spot-check) before training is as important as sourcing it — assuming a downloaded dataset is “ready” without checking wastes time later if problems surface mid-training.
- Documenting a dataset transparently (what was sourced vs. self-collected, exact split numbers, class balance) is more defensible for evaluation than presenting a polished but inaccurate account — and extending a public base with genuinely self-collected images closes that gap rather than just explaining it away.
- Small workflow issues (a failed download, a changed UI button) can cost real time — it's worth double-checking file integrity immediately after each download rather than assuming success.
- Class imbalance and small-object performance are worth flagging before training begins, not after seeing disappointing metrics.

6. Week 3 Goals

Week 3 shifts the project from a static per-frame detector into a real-time video analytics system — the model must track riders across frames, not just detect them in isolation. Goals below are scoped specifically to the helmet detection use case.

- Integrate a multi-object tracker (e.g., ByteTrack) on top of the trained YOLOv8 detector so each rider is assigned a persistent, unique ID across consecutive frames rather than being re-detected independently every frame.
 - Implement line-crossing or zone-based counting logic to tally total riders passing through a defined region of the video, avoiding duplicate counts of the same rider across frames.
 - Track per-ID helmet compliance over the duration each rider remains in frame, rather than a single-frame snapshot, to reduce false compliance flips caused by momentary occlusion or misclassification.
 - Log basic movement analytics per tracked ID (entry/exit time, direction of travel, dwell time in frame) to support downstream trend analysis.
 - Build a lightweight live overlay (bounding box + ID + compliance status) plus a running counter for total riders and helmet-compliance rate, rendered on the video stream.
 - Generate an automated end-of-session summary report (CSV or PDF) with total riders counted, compliant vs. non-compliant counts, and compliance rate, to demonstrate the system as a functioning analytics pipeline rather than a standalone detector.
-